Feynman Diagrams and AI: A Survey of Field-Theoretic Methods in Deep Learning

hy-wu

May 13, 2026

1 Introduction

1.1 The Gap Between Math and Code

Deep learning has achieved remarkable empirical success across domains, yet the relationship between the mathematical structures that underpin neural networks and the imperative code that implements them remains fundamentally fractured. Practitioners design architectures through a combination of intuition and trial-and-error, while theoretical insights—from neural tangent kernels to scaling laws—often arrive after the fact, explaining rather than guiding construction. This gap between mathematical formalism and computational practice imposes real costs: it slows the translation of theoretical advances into working systems, makes architectural innovations difficult to reason about compositionally, and forces compiler optimizations to rediscover structural properties that are obvious at the mathematical level.

The problem is not unique to deep learning. In physics, the same gap existed in the early days of quantum field theory, where increasingly complex calculations demanded a more powerful representational language. The solution came in the form of Feynman diagrams—a diagrammatic calculus that made perturbative calculations tractable by encoding the structure of interactions directly into graphical form. What made Feynman diagrams revolutionary was not merely their computational utility, but their ability to serve simultaneously as a mathematical formalism with precise rules, an intuitive visual language for reasoning about interactions, and a concrete prescription for computation.

This survey argues that a similar diagrammatic revolution is possible—and necessary—for deep learning infrastructure. By adopting the Feynman diagram as a unifying representation that spans from high-level architecture design down to low-level compiler optimizations, we can bridge the gap between mathematical structure and executable code.

1.2 The Diagrammatic Vision

The central thesis of this survey is that Feynman diagrams, and the broader family of diagrammatic formalisms they inspire, offer a revolutionary way to represent, analyze, compile, and optimize neural network architectures. The core mapping is straightforward yet profound: in a Feynman diagram, external lines correspond to particles entering or leaving an interaction; in a neural network, input and output tokens play

the same role. Vertices represent local interactions—attention operations, nonlinearities, or weight multiplications—while loops capture recurrent or iterative dynamics. The diagram as a whole represents a composition of interactions that can be evaluated, differentiated, and optimized.

Modern deep learning infrastructure spans multiple levels of abstraction: high-level architecture descriptions in frameworks like PyTorch and JAX, intermediate representations in compilers like MLIR and TVM, and low-level GPU kernel implementations. At each level, a different representation language is used, and the transformations between them are ad hoc and lossy. The diagrammatic vision proposes a unified representation that flows through all these levels, enabling formal reasoning about equivalence, optimization, and compilation within a single framework.

This vision draws together ideas from quantum field theory (Feynman diagrams, path integrals, renormalization), category theory (string diagrams, monoidal categories), statistical physics (partition functions, maximum entropy), and compiler design (intermediate representations, graph rewriting, equality saturation). Each discipline contributes a piece of the puzzle: physics provides the diagrammatic calculus, category theory provides compositional semantics, statistical physics connects to learning theory, and compiler engineering provides the path to efficient execution.

1.3 Scope and Outline

The survey is organized into five core chapters following this introduction. Chapter 2 establishes the physical and mathematical foundations: Feynman diagrams and quantum field theory, tensor networks and their diagrammatic algebra, categorical diagrammatic languages, and additional tools including Clifford algebras, p-adic numbers, and density matrices. Chapter 3 develops the mapping between neural networks and field theories, covering the neural tangent kernel, large-N expansion, diagrammatic theories of deep networks, and model architectures viewed as Feynman diagrams. Chapter 4 treats attention mechanisms through the lens of statistical physics and path integrals. Chapter 5 surveys the deep learning compilation stack—intermediate representations, tensor compilation, graph rewriting, and architecture search—identifying opportunities for diagrammatic abstraction. Chapter 6 synthesizes these threads into a concrete vision for a unified Feynman Diagram Compiler, and Chapter 7 concludes with open problems and a research roadmap.

2 Foundations: Physical and Mathematical Toolbox

2.1 Feynman Diagrams and Quantum Field Theory

Feynman diagrams originated as a perturbative tool in quantum electrodynamics, providing a graphical representation of terms in the Dyson expansion of the S-matrix [21, 47]. In their canonical form, each diagram corresponds to a definite integral expression for a scattering amplitude: external lines represent incoming and outgoing particles, internal lines represent propagators, and vertices represent interaction terms in the Lagrangian. The diagrammatic expansion is systematic—each order in the coupling constant corresponds to diagrams with a fixed number of vertices—and the rules for translating diagrams to integrals (the Feynman rules) are derived directly from the underlying field theory.

Three features of Feynman diagrams are particularly relevant for establishing correspondences with neural network computations. First, compositionality: complex diagrams are built from simpler components following precise combinatorial rules, enabling systematic approximation schemes analogous to building deep networks from layers. Second, equivalence relations: different diagram topologies can represent the same physical process due to underlying symmetries, providing a rich set of rewrite rules that mirror algebraic identities in tensor computation. Third, renormalization: divergences in loop diagrams are absorbed into physically observable parameters through a systematic procedure that reveals how effective descriptions change across scales, finding a natural analogue in the depth-wise hierarchy of representations in deep networks.

The connection between Feynman diagrams and computational graphs has been made explicit in recent work. Hou et al. [24] demonstrate that Feynman diagrams for many-electron field theories can be represented as computational graphs, enabling the application of graph-based machine learning and high-performance computing tools to quantum many-body problems. Mitchell et al. [32] further show that graph neural networks can learn to evaluate Feynman diagrams directly, providing a proof of concept for diagram-to-computation mapping.

2.2 Tensor Networks and Diagrammatic Algebra

Tensor networks provide a diagrammatic language for factorizing high-dimensional tensors into networks of contractions between lower-order tensors, originally developed in quantum many-body physics. Each tensor is represented as a node with legs corresponding to its indices; connecting legs represents contraction over shared indices. The canonical architectures include matrix product states (MPS) for one-dimensional chains, projected entangled pair states (PEPS) for two-dimensional lattices, and tree tensor networks for hierarchical structure.

The connection to neural networks is bidirectional. Tensorized embeddings compress large embedding tables via tensor decomposition, while tensorized attention mechanisms approximate the full attention matrix via low-rank tensor structure. The diagrammatic calculus of tensor networks—where contracting a shared index corresponds to summing over a common dimension—provides a formal language for reasoning about information flow in neural architectures. Berman and colleagues [7] explicitly develop a quantum field theory approach to encoding data that bridges tensor networks and neural network architectures.

2.3 Diagrammatic Languages for Model Architecture

Beyond tensor networks, string diagrams from category theory provide a fully formal graphical calculus for composing processes in symmetric monoidal categories [30]. Objects represent data types and morphisms represent computations; sequential composition connects outputs to inputs, while parallel composition places processes side by side. The theory of string diagram rewriting [3] provides formal foundations for equational reasoning about diagrammatic computations.

The categorical approach has been implemented in practical systems. DisCoPy [46] provides a Python library for constructing and manipulating string diagrams, supporting both quantum and classical computation. The DisCoCat framework applies categorical compositional semantics to natural language processing [41, 26]. For deep learning

specifically, neural circuit diagrams [2] offer a robust visual language with precise semantics that can be translated to executable code in frameworks like PyTorch. These formalisms differ in expressive power, ranging from informal notations to fully formal categorical calculi with complete equational theories [8, 9].

2.4 The Physical Mathematician's Toolbox

Three additional mathematical structures enrich the diagrammatic framework beyond its core foundations. First, geometric (Clifford) algebra provides a unified language for representing geometric transformations, rotations, and equivariances in neural networks [40]. The Geometric Algebra Transformer and Clifford Group Equivariant Neural Networks demonstrate that this formalism naturally captures Lorentz symmetries relevant to physics applications and rotational equivariances relevant to 3D data processing [43, 11].

Second, p-adic numbers and ultrametric geometry offer a natural formalism for hierarchical structure in language and other tree-structured data [33]. The strong triangle inequality $\|x - z\| \leq \max(\|x - y\|, \|y - z\|)$ captures the hierarchical organization of linguistic structures, and ultrametric attention mechanisms can exploit this to achieve $O(N \log N)$ complexity.

Third, density matrices from quantum mechanics provide a formalism for representing uncertainty and ambiguity in neural representations. This perspective enriches the diagrammatic language by treating token representations not as vectors but as density operators, with applications in quantum natural language processing [31].

3 Neural Network Theory Through the Feynman Lens

3.1 Neural Networks as Field Theories

The mapping between neural networks and statistical field theories is one of the most productive developments in theoretical deep learning. In the infinite-width limit, a neural network with i.i.d. initialized parameters converges to a Gaussian process (NNGP), and its training dynamics under gradient descent are governed by the neural tangent kernel (NTK). Both the NNGP and NTK are Gaussian theories—the field-theoretic analogues of free theories—where all correlations are determined by two-point functions and higher-order correlations factorize via Wick's theorem [34].

Finite-width corrections introduce interactions that are captured by higher-order Feynman diagrams [23]. These corrections are organized by the large-N expansion, where N is the hidden layer width. The leading order corresponds to planar diagrams (the GP/NTK limit), the next order captures fluctuation effects through loop diagrams, and successive orders systematically incorporate more complex interactions. The effective field theory (EFT) approach extends this framework to deep architectures [6], where depth becomes a renormalization group direction and irrelevant operators are suppressed as information propagates through layers. This perspective explains neural scaling laws through the lens of large-N field theory [35]. Dynamical mean-field theory [45] provides a complementary approach specific to self-attention networks, capturing the correlated dynamics of tokens in transformers.

3.2 Diagrammatic Theory of Deep Neural Networks

Building on the field-theoretic foundation, we develop a fully diagrammatic representation of neural network computations [17]. Each neural network layer maps to a vertex in a Feynman-like diagram: a fully-connected layer is a rank-2 vertex contracting input and weight indices; a convolutional layer is a locally-connected vertex with tied weights; a nonlinear activation function acts as an interaction vertex coupling multiple field components. The categorical foundations of gradient-based learning [18] provide formal semantics for this mapping, relating backpropagation to the reverse derivative construction in Cartesian differential categories.

The diagrammatic representation reveals a striking duality: the forward pass and backward pass (backpropagation) correspond to a diagram and its Hermitian conjugate. This duality suggests that forward and backward passes can be fused and optimized as a single diagrammatic unit, rather than treated as separate computational phases—an insight with practical implications for compiler optimization that we develop further in Chapter 5.

3.3 Model Architectures as Feynman Diagrams

We examine specific neural network architectures through the lens of diagrammatic patterns [2]. Convolutional networks correspond to locally-connected vertices with weight sharing, where the connectivity pattern defines a translation-equivariant diagram fragment. The convolution operation appears as a structured contraction representable as a tensor network with sparse connectivity, and pooling corresponds to a coarse-graining transformation analogous to the renormalization group.

Transformers combine pairwise attention vertices—rank-4 tensors connecting all pairs of tokens—with skip connections. The multi-head mechanism decomposes the rank-4 attention vertex into a sum of lower-rank components, analogous to tensor network decomposition. The enriched category theory of language [10] provides a formal framework for understanding how linguistic structure shapes these architectural choices, relating syntactic composition to diagram connectivity.

The key insight is that architectural innovation becomes a process of composing diagram fragments rather than engineering imperative code. An attention mechanism is a particular vertex topology; a residual connection is a diagrammatic bypass structure; layer normalization is a constraint on field normalization. This shift from imperative code to declarative diagrams opens the door to automated architecture generation and optimization through diagram rewriting.

3.4 Field-Theoretic Perspectives

The identification of computational graphs with Feynman diagrams opens up rich theoretical tools beyond representation. Diagram topology directly determines computational complexity: planar diagrams correspond to sequential feedforward computation, loop diagrams to recurrent computation, and crossed diagrams to non-local interactions requiring attention. The geometric field theory framework for transformers [1] provides a unified mathematical structure that captures attention, layer normalization, and residual connections as geometric objects in a field-theoretic setting.

The renormalization group perspective provides a unified explanation for scaling laws, where scaling exponents are determined by relevant and irrelevant operators of

the effective theory. Efficient learning of exponential family distributions [42] connects the statistical structure of attention to the broader theory of maximum-entropy models. Diagrammatic invariants—quantities preserved under rewrites, analogous to conservation laws in physics—offer design principles for constructing new architectures with guaranteed properties.

4 Attention Mechanisms and Path Integrals

4.1 Path Integral Formulation of Attention

The path integral formulation of quantum mechanics, where transition amplitudes are obtained by summing over all possible trajectories weighted by the exponential of the action, provides a natural language for attention mechanisms. The integral transformer [25] and folded context condensation [36] cast transformer forward passes as path integrals over token trajectories, where the attention score emerges from summing over all possible pairwise token interaction paths with softmax playing the role of the Boltzmann factor $\exp(-\beta E)$.

Graph field integrators [16] generalize this connection, showing that the path integral framework applies not only to attention but to a broad class of graph-structured computations in deep learning. This suggests that the diagrammatic compiler vision—where computational graphs are manipulated as physical systems—has a rigorous foundation in the path integral formalism.

4.2 Attention as Statistical Physics

The softmax function is the maximum-entropy distribution over token interactions given expected alignment scores [22]. The attention matrix becomes a partition function, and self-attention computes a form of free energy minimization: $F_i = -\frac{1}{\beta} \log Z_i = \langle E_i \rangle - \frac{1}{\beta} S_i$. Temperature scaling controls the sharpness of attention, with zero temperature recovering hard attention and infinite temperature giving uniform averaging. The maximum entropy softmax policy gradient framework [15] provides a reinforcement learning perspective on this thermodynamic view, connecting attention mechanisms to entropy-regularized optimization.

5 Deep Learning Compilation and Optimization Infrastructure

5.1 Intermediate Representations for Deep Learning

The compilation of deep learning models relies on intermediate representations (IRs) bridging framework code and hardware instructions. MLIR [28] introduced a dialect-based infrastructure enabling progressive lowering through multiple abstraction levels. The TVM stack [14] pioneered the separation between tensor expression and schedule, while Relay [39] provides a functional-style IR with first-class control flow. Halide [38] introduced the foundational insight of decoupling algorithm from schedule that influenced the entire DL compiler landscape. Each IR occupies a different point in the

expressiveness-compilability trade-off. An ideal diagrammatic IR would combine categorical string diagram compositionality with MLIR's multi-level lowering, representing computations as composable diagram fragments with formal semantics.

5.2 Compilation and Auto-Optimization

Modern DL compilers decompose optimization into tensor expression lowering, kernel fusion, memory hierarchy optimization, and auto-scheduling. TVM [14] pioneered the separation between expression and schedule; Ansor [52] extended this with template-free search using a learned cost model; OpenTuner [5] provided the foundational multi-armed bandit approach to auto-tuning. Nautilus [51] advances GPU kernel auto-scheduling with efficient tiled execution. These techniques map naturally onto diagrammatic compilation: each diagram fragment corresponds to a tensor operation, operator fusion becomes diagram contraction, memory planning corresponds to tensor placement, and auto-tuning becomes search over diagram decompositions structured by rewrite rules.

5.3 Graph Rewriting and Equality Saturation

Equality saturation [44] uses e-graphs to compactly represent large spaces of equivalent programs by sharing common subexpressions. The egg library provides efficient implementation; Metatheory.jl [13] extends equality saturation to Julia with extensible algebraic computation; sketch-guided equality saturation [27] scales the technique to complex optimizations of functional programs. Datalog-based approaches [50] unify equality saturation with relational query processing for improved scalability.

For tensor programs specifically, Mirage [48] uses a multi-level superoptimizer that discovers GPU kernels outperforming hand-tuned implementations. Prism [49] extends this to symbolic superoptimization using cost-guided program synthesis. Diagrammatic representations are naturally suited for equality saturation because Feynman diagram identities—Wick contraction, crossing symmetry, the optical theorem—correspond directly to tensor graph rewrite rules, reducing the search space while capturing a wider range of transformations than manually engineered optimization passes.

5.4 Neural Architecture Search

Neural architecture search (NAS) automates model design, evolving from RL-based approaches [53] through differentiable methods (DARTS, ENAS [37]) and proxyless approaches [12] to hardware-aware search [20, 19]. A diagrammatic grammar, where architectures are generated by composing diagram fragments according to well-defined production rules, would provide a more structured search space. This perspective unifies NAS with superoptimization and equality saturation, suggesting the same rewrite engine can drive architecture-level and operator-level optimization jointly—a capability impossible when architecture search and compiler optimization are performed independently. The categorical foundations surveyed in [17] provide the formal underpinnings for such a unified optimization framework.

6 Toward a Unified Feynman Diagram Compiler

6.1 Compiling Diagrammatic Representations to GPU

Compiling diagrammatic representations to GPU code presents four challenges. First, diagram-to-tensor mapping: vertices must be lowered to efficient tensor operations, with automatic fusion guided by diagram topology. EC-SpMM [29] demonstrates efficient compilation of sparse matrix patterns on GPUs, relevant for mapping sparse diagram structures to hardware. Second, dynamic graph structures: modern architectures employ data-dependent connectivity requiring JIT compilation. Third, hardware-specific optimization: GPU architectures differ in memory hierarchy and tensor core capabilities. Fourth, multi-granularity optimization: decisions at architecture, operator, and instruction levels are interdependent, and the diagrammatic framework's multi-level representation supports coordinated optimization across all scales through a single rewrite engine.

6.2 A Unified Diagrammatic Compiler for AI

We propose a Feynman Diagram Compiler (FDC) with four components. First, a diagrammatic IR with formal equational theory, built on the categorical foundations of string diagrams [46] and extended with tensor-specific operations, supporting multiple abstraction levels from high-level architectural motifs to low-level memory access patterns. Second, an equality saturation rewrite engine that applies rules from the equational theory, guided by a cost model for both computational efficiency and hardware constraints. Third, categorical semantics mapping diagram fragments to tensor operations [17], ensuring rewrites preserve computational semantics. Fourth, a code generation backend using auto-tuning, building on the multi-level IR infrastructure of Relax [28] and TVM [14]. The FDC operates in ahead-of-time (AOT) mode for static diagrams and just-in-time (JIT) mode for dynamic architectures.

6.3 Cross-Domain Analogies as Inductive Biases

The diagrammatic perspective reveals analogies across scientific domains, enabling systematic transfer of inductive biases. The agentic diagrammatica framework [4] demonstrates autonomous symbolic computation in high-energy physics that parallels compiler optimization for deep learning graphs. Monte Carlo methods in statistical physics, which estimate partition functions by sampling, share structure with stochastic computation graphs. Computational fluid dynamics models transport through continuous media, paralleling information flow in vision-language models. Large-scale structure formation in cosmology—amplification of certain modes and suppression of others—parallels latent representation formation in generative models. The diagrammatic language provides a shared formal grammar across these domains, making cross-domain analogies precise and actionable.

7 Conclusion and Outlook

7.1 Open Problems

We identify five open problems. First, developing complete categorical semantics for transformer architectures—while string diagrams provide a general framework, the specific structure of attention (softmax normalization, causal masking, multi-head decompositions) remains unformalized. The survey by Crescenzi et al. [17] provides a comprehensive overview of the open challenges in categorical deep learning. Second, proving correctness of diagram rewrite rules for tensor programs, requiring a formal framework relating diagrammatic transformations to tensor algebra. Third, scaling equality saturation [44] to production-scale models with thousands to millions of operations, requiring hierarchical decomposition and incremental techniques. Fourth, designing benchmark suites for diagrammatic compilation across diverse architectures and hardware targets. Fifth, understanding the theoretical limitations of diagrammatic representations—which computations are efficiently representable and the complexity of diagram equivalence problems.

7.2 The Diagram is the Code

The Feynman diagram is not merely a theoretical tool for understanding neural networks, but a practical foundation for next-generation AI infrastructure. Just as Feynman diagrams revolutionized quantum field theory by making complex calculations manageable and intuitive, a diagrammatic foundation for deep learning can transform how we design, analyze, compile, and optimize neural networks. We have shown that the mapping between Feynman diagrams and neural network computations is a substantive formal correspondence: physical mathematics provides structures—path integrals, tensor networks, string diagrams—that map onto computational patterns, and compiler engineering provides the tools—IRs, graph rewriting, auto-tuning—to translate these structures into efficient code. The ultimate promise: a researcher draws a Feynman-like diagram describing their architecture, and a compiler generates optimal, hardware-efficient code. The diagram is the code, and the code is the diagram.

References

[1] *A Unified Geometric Field Theory Framework for Transformers*. arXiv. 2025.

[2] Vincent Abbott. *Neural Circuit Diagrams: Robust Diagrams for the Communication, Implementation, and Analysis of Deep Learning Architectures*. arXiv (Cornell University). 2024. DOI: 10.48550/arxiv.2402.05424.

[3] Matteo Acclavio. "String diagram rewriting : applications in category and proof theory". In: (). DOI: 10.70675/83f48b85zdd67z4e09zba83zb54adcaaaa6e.

[4] *Agentic Diagrammatica: Towards Autonomous Symbolic Computation in High Energy Physics*. arXiv. 2026.

[5] Jason Ansel et al. "OpenTuner". In: (2014). DOI: 10.1145/2628071.2628092.

[6] Ian Banta. *Structures of neural network effective theories*. Physical Review D. 2024. DOI: 10.1103/PhysRevD.109.105007.

[7] David S. Berman, Marc S. Klinger, and Alexander G. Stapleton. *NCoder — A Quantum Field Theory Approach to Encoding Data*. Machine Learning: Science and Technology. 2024. DOI: 10.1088/2632-2153/ade04c.

[8] F. Bonchi et al. "String Diagram Rewrite Theory I: Rewriting with Frobenius Structure". In: *Journal of the ACM (JACM)* 69 (2020), pp. 1 –58.

[9] F. Bonchi et al. "String diagram rewrite theory II: Rewriting with symmetric monoidal structure". In: *Mathematical Structures in Computer Science* 32 (2021), pp. 511 –541.

[10] Tai-Danae Bradley, John Terilla, and Yiannis Vlassopoulos. "An Enriched Category Theory of Language: From Syntax to Semantics". In: *La Matematica* (2022). DOI: 10.1007/s44007-022-00021-2.

[11] Johann Brehmer et al. *A Lorentz-Equivariant Transformer for All of the LHC*. arXiv (Cornell University). 2024. DOI: 10.48550/arxiv.2411.00446.

[12] Han Cai, Ligeng Zhu, and Song Han. *ProxylessNAS: Direct Neural Architecture Search on Target Task and\{}n Hardware*. arXiv (Cornell University). 2018. DOI: 10.48550/arxiv.1812.00332.

[13] Alessandro Cheli. "Metatheory.jl: Fast and Elegant Algebraic Computation in Julia with Extensible Equality Saturation". In: *Journal of Open Source Software* (2021). DOI: 10.21105/joss.03078.

[14] Tianqi Chen et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. arXiv (Cornell University). 2018. DOI: 10.48550/arxiv.1802.04799.

[15] Jean Seong Bjorn Choe and Jong kook Kim. "Maximum Entropy Softmax Policy Gradient via Entropy Advantage Estimation". In: *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence* (2025). DOI: 10.24963/ijcai.2025/552.

[16] Krzysztof Choromanski. "Graph Field Integrators: From Transformers to Feynman Path Integrals". In: *Princeton Discrete Math Seminar (Jan 2026)* (2026).

[17] Francesco Riccardo Crescenzi. "Towards a Categorical Foundation of Deep Learning: A Survey". In: *AMS Degree Thesis (University of Bologna)* (2024). DOI: 10.48550/arxiv.2410.05353.

[18] G. S. H. Cruttwell et al. "Categorical Foundations of Gradient-Based Learning". In: *Lecture notes in computer science* (2022). DOI: 10.1007/978-3-030-99336-8_1.

[19] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. "Neural Architecture Search". In: ☒*The* ☒*Springer series on challenges in machine learning* (2019). DOI: 10.1007/978-3-030-05318-5_3.

[20] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. *Neural Architecture Search: A Survey*. arXiv (Cornell University). 2018. DOI: 10.48550/arxiv.1808.05377.

[21] Richard P. Feynman and James M. Cline. *Feynman Lectures on the Strong Interactions*. 2020.

[22] Diego Granziol et al. *MEMe: An Accurate Maximum Entropy Method for Efficient Approximations in Large-Scale Machine Learning*. MEMe: An Accurate Maximum Entropy Method for Efficient Approximations in Large-Scale Machine Learning. Entropy, 21(6), 551 (2019). 2019. DOI: 10.3390/e21060551.

[23] Max Guillen, Philipp Misof, and Jan E. Gerken. *Finite-Width Neural Tangent Kernels from Feynman Diagrams*. arXiv. 2025. DOI: 10.48550/arXiv.2508.11522.

[24] Pengcheng Hou et al. *Feynman Diagrams as Computational Graphs*. arXiv. 2024. DOI: 10.48550/arXiv.2403.18840.

[25] *Integral Transformer: Denoising Attention, Not Too Much Not Too Little*. arXiv. 2025.

[26] Dimitri Kartsaklis. *Coordination in Categorical Compositional Distributional Semantics*. EPTCS 221, 2016, pp. 29-38. 2016. DOI: 10.4204/eptcs.221.4.

[27] Thomas Koehler, Phil Trinder, and Michel Steuwer. *Sketch-Guided Equality Saturation: Scaling Equality Saturation to Complex Optimizations of Functional Programs*. 2021.

[28] Ruihang Lai et al. *Relax: Composable Abstractions for End-to-End Dynamic Machine Learning*. arXiv (Cornell University). 2023. DOI: 10.48550/arxiv.2311.02103.

[29] J. Lin et al. "EC-SpMM: Efficient Compilation of SpMM Kernel on GPUs". In: (2023). DOI: 10.1145/3605573.3605632.

[30] Dan Marsden. *Category Theory Using String Diagrams*. arXiv (Cornell University). 2014. DOI: 10.48550/arxiv.1401.7220.

[31] Konstantinos Meichanetzidis et al. "Quantum Natural Language Processing on Near-Term Quantum Computers". In: *Electronic Proceedings in Theoretical Computer Science* (2021). DOI: 10.4204/eptcs.340.11.

[32] Harrison Mitchell, Alexander Norcliffe, and Pietro Liò. *Learning Feynman Diagrams using Graph Neural Networks*. NeurIPS ML4PS 2022. 2022.

[33] Fionn Murtagh. *From Data to the p-Adic or Ultrametric Model*. p-Adic Numbers, Ultrametric Analysis and Applications, 1, 58-68, 2009. 2008. DOI: 10.1134/s2070046609010063.

[34] *Neural network field theories: non-Gaussianity, actions, and locality*. Machine Learning: Science and Technology. 2024. DOI: 10.1088/2632-2153/ad17d3.

[35] *Neural scaling laws from large-N field theory*. Machine Learning: Science and Technology. 2025. DOI: 10.1088/2632-2153/adc872.

[36] Won-Gi Paeng et al. *Folded Context Condensation in Path Integral Formalism for Infinite Context Transformers*. IEEE Access. 2025. DOI: 10.1109/ACCESS.2025.3574857.

[37] Hieu Pham et al. "Efficient Neural Architecture Search via Parameter Sharing". In: *ArXiv* abs/1802.03268 (2018).

[38] Jonathan Ragan-Kelley et al. "Halide". In: *Communications of the ACM* (2017). DOI: 10.1145/3150211.

[39] Jared Roesch et al. "Relay: A High-Level IR for Deep Learning". In: *ArXiv* abs/1904.08368 (2019).

[40] David Ruhe et al. *Geometric Clifford Algebra Networks*. arXiv (Cornell University). 2023. DOI: 10.48550/arxiv.2302.06594.

[41] Mehrnoosh Sadrzadeh. *Quantifier Scope in Categorical Compositional Distributional Semantics*. EPTCS 221, 2016, pp. 49-57. 2016. DOI: 10.4204/eptcs.221.6.

[42] Abhin Shah, Devavrat Shah, and Gregory W. Wornell. *On Computationally Efficient Learning of Exponential Family Distributions*. 2023.

[43] Jonas Spinner et al. *Lorentz-Equivariant Geometric Algebra Transformers for High-Energy Physics*. arXiv (Cornell University). 2024. DOI: 10.48550/arxiv.2405.14806.

[44] Ross Tate et al. "Equality Saturation: A New Approach to Optimization". In: *Logical Methods in Computer Science* (2011). DOI: 10.2168/lmcs-7(1:10)2011.

[45] Lorenzo Tiberi et al. *Dynamical Mean-Field Theory of Self-Attention Neural Networks*. arXiv. 2024.

[46] Alexis Toumi, Giovanni de Felice, and Richie Yeung. *DisCoPy for the quantum computer scientist*. arXiv (Cornell University). 2022. DOI: 10.48550/arxiv.2205.05190.

[47] Stefan Weinzierl. *Feynman Integrals*. 2022.

[48] Mengdi Wu et al. *Mirage: A Multi-Level Superoptimizer for Tensor Programs*. arXiv (Cornell University). 2024. DOI: 10.48550/arxiv.2405.05751.

[49] Mengdi Wu et al. *Prism: Symbolic Superoptimization of Tensor Programs*. 2026.

[50] Yihong Zhang et al. "Better Together: Unifying Datalog and Equality Saturation". In: *Proceedings of the ACM on Programming Languages* (2023). DOI: 10.1145/3591239.

[51] Yifan Zhao et al. *Nautilus: An Auto-Scheduling Tensor Compiler for Efficient Tiled GPU Kernels*. ArXiv.org. 2026.

[52] Lianmin Zheng et al. *Ansor: Generating High-Performance Tensor Programs for Deep Learning*. arXiv (Cornell University). 2020. DOI: 10.48550/arxiv.2006.06762.

[53] Barret Zoph and Quoc V. Le. *Neural Architecture Search with Reinforcement Learning*. arXiv (Cornell University). 2016. DOI: 10.48550/arxiv.1611.01578.